BRAIN TUMOR SEGMENTATION

Submitted By:	**SMRITI KUMARI**
Roll No.:	24155428
Section:	IPA-9
Department:	Department of Computer Science and Engineering

1. ABSTRACT

Brain tumors represent one of the most severe and life-threatening medical conditions, necessitating precise diagnostic evaluation. Magnetic Resonance Imaging (MRI) is widely utilized to examine brain tissues and identify structural abnormalities. In this project, I implemented an automated brain MRI abnormality segmentation pipeline leveraging a deep learning model based on the U-Net architecture.

The system accepts a compatible multi-channel MRI slice, performs standard image preprocessing and channel-wise normalization, and processes the tensor through the convolutional network. The model computes a continuous probability map indicating region abnormalities, which is subsequently converted into a binary segmentation mask using thresholding. The results are rendered and visual overlays are generated for spatial verification.

2. INTRODUCTION

Medical image segmentation involves partitioning a digital image into meaningful anatomical regions to isolate structures of interest. In neuro-oncology, standard manual segmentation of brain MRI scans is time-consuming and subject to inter-observer variability. Deep learning models, specifically Convolutional Neural Networks (CNNs), have significantly improved automated semantic segmentation tasks.

The U-Net architecture is specifically engineered for biomedical image segmentation, featuring a symmetric encoder-decoder structure with skip connections that preserve spatial granularity across resolution scales. This project documents my practical implementation of a U-Net segmentation workflow using PyTorch, demonstrating effective pixel-wise classification on brain MRI datasets.

3. OBJECTIVES

- To explore deep learning techniques applied to medical image analysis and semantic segmentation.
- To construct and load a trained U-Net neural network architecture within the PyTorch framework.
- To design an end-to-end data preprocessing pipeline for standardizing input MRI brain slices.
- To generate continuous abnormality probability heatmaps and derive binary segmentation masks.
- To visualize predictions using Matplotlib and construct clear visual overlays onto original images.
- To export high-resolution segmentation masks and quantitative evaluation visual outputs.

4. TECHNOLOGIES & TOOLS USED

Technology / Library	Role in Implementation
Python 3.x	Core programming language used for script execution and data manipulation.
Google Colab	Interactive cloud execution environment with GPU acceleration support.
PyTorch & Torchvision	Deep learning library utilized for network loading, tensor operations, and inference.
U-Net Architecture	Convolutional network configured with 3 input channels and 1 output probability channel.
NumPy	Used for matrix transformations, array manipulation, and threshold calculations.
Pillow (PIL)	Handling image ingestion, color format conversions, and saving output masks.
Matplotlib	Plotting visual figures, rendering probability maps, and generating comparative plots.

5. PROPOSED METHODOLOGY

1. **Data Acquisition:** Ingest a 3-channel brain MRI slice formatted across standard structural sequences.
2. **Preprocessing:** Resize images to a unified resolution of 256×256 pixels and execute per-channel mean and standard deviation standardization.
3. **Model Initialization:** Load the U-Net architecture weights into memory and transition the network to evaluation mode.
4. **Forward Inference:** Pass the standardized tensor through the network without gradient calculation to output pixel probabilities.
5. **Mask Generation:** Apply a decision threshold (T = 0.5) on the probability map to produce a binary mask.
6. **Visualization & Export:** Construct visual overlays and save final output images to the local runtime.

6. U-NET ARCHITECTURE DESIGN

The U-Net architecture consists of a contracting path (encoder) and an expansive path (decoder). The encoder follows a traditional convolutional network layout, applying repeated 3×3 convolutions followed by rectified linear units (ReLU) and 2×2 max pooling operations to extract feature hierarchies while downsampling spatial dimensions.

The decoder path upsamples feature maps using 2×2 transpose convolutions, concatenating them with matching high-resolution feature maps from the contracting path via skip connections. This design allows the network to retain fine spatial localization details essential for accurate anatomical segmentation.

7. SYSTEM WORKFLOW

Input MRI Slice → Preprocessing & Normalization → U-Net Inference → Probability Map → Thresholding (T=0.5) → Binary Mask & Overlay

8. IMPLEMENTATION AND PYTHON SOURCE CODE

The practical implementation was executed in Google Colab. Below is the structured Python source code written and executed for this project.

Step 1: Environment Setup and Library Imports

```
!pip install torch torchvision matplotlib pillow numpy import torch import numpy as np
import matplotlib.pyplot as plt import urllib.request from PIL import Image from
torchvision import transforms from google.colab import files print("Environment
configured successfully. PyTorch Version:", torch.__version__)
```

This initial cell installs required packages and imports essential libraries for neural network inference, array handling, and visualization.

Step 2: Loading Model Architecture and Weights

```
model = torch.hub.load( 'mateuszbuda/brain-segmentation-pytorch', 'unet',
in_channels=3, out_channels=1, init_features=32, pretrained=True ) model.eval() device
= torch.device("cuda" if torch.cuda.is_available() else "cpu") model = model.to(device)
print("U-Net Model Loaded Successfully") print("Execution Device:", device)
```

The network is configured to accept 3 input channels and generate a single output channel corresponding to tumor abnormality likelihood. Evaluating on GPU significantly reduces inference latency.

Step 3: MRI Image Download and Ingestion

```
url = ( "https://github.com/mateuszbuda/" "brain-segmentation-pytorch/raw/master/"
"assets/TCGA_CS_4944.png" ) urllib.request.urlretrieve(url, "brain_mri.png") image=
Image.open("brain_mri.png").convert("RGB") print("MRI Image Ingested Successfully")
print("Input Resolution:", image.size)
```

Step 4: Input Image Inspection

```
plt.figure(figsize=(5, 5)) plt.imshow(image) plt.title("Original Brain MRI Slice")
plt.axis("off") plt.show()
```

Step 5: Image Preprocessing and Normalization

```
image_array = np.asarray(image).astype(np.float32) # Compute per-channel mean and
standard deviation mean = image_array.mean(axis=(0, 1)) std = image_array.std(axis=(0,
1)) std[std == 0] = 1.0 preprocess = transforms.Compose([ transforms.ToTensor(),
transforms.Normalize(mean=mean.tolist(), std=std.tolist()) ]) input_tensor =
preprocess(image) input_batch = input_tensor.unsqueeze(0).to(device)
print("Preprocessing Pipeline Executed") print("Batch Tensor Dimensions:",
input_batch.shape)
```

The image data is transformed into a floating-point PyTorch tensor and standardized across channels before adding a batch dimension to match network input requirements.

Step 6: Executing Deep Learning Inference

```
with torch.no_grad(): output = model(input_batch) # Extract probability map from
network output probability_map = output[0, 0].cpu().numpy() threshold = 0.5
segmentation_mask = (probability_map > threshold).astype(np.uint8) print("Segmentation
Inference Completed") print("Output Probability Map Shape:", probability_map.shape)
print("Unique Binary Mask Values:", np.unique(segmentation_mask))
```

Step 7: Plotting Segmentation Pipeline Visuals

```
plt.figure(figsize=(15, 5)) plt.subplot(1, 3, 1) plt.imshow(image) plt.title("Original
MRI") plt.axis("off") plt.subplot(1, 3, 2) plt.imshow(probability_map, cmap="hot")
plt.title("Probability Map") plt.axis("off") plt.subplot(1, 3, 3)
plt.imshow(segmentation_mask, cmap="gray") plt.title("Binary Mask (T=0.5)")
plt.axis("off") plt.tight_layout() plt.show()
```

Step 8: Constructing Segmented Region Overlay

```
image_np = np.asarray(image).copy() overlay = image_np.copy() # Apply red overlay on
positive mask regions overlay[segmentation_mask == 1] = ( 0.5 *
image_np[segmentation_mask == 1] + 0.5 * np.array([255, 0, 0]) ).astype(np.uint8)
plt.figure(figsize=(12, 5)) plt.subplot(1, 2, 1) plt.imshow(image_np)
plt.title("Original Input") plt.axis("off") plt.subplot(1, 2, 2) plt.imshow(overlay)
plt.title("Tumor Segmentation Overlay") plt.axis("off") plt.tight_layout() plt.show()
```

Step 9: Result Exportation

```
Image.fromarray(segmentation_mask * 255).save("segmentation_mask.png")
Image.fromarray(overlay).save("tumor_overlay.png") print("Output Files Saved
Successfully to Local Storage") files.download("segmentation_mask.png")
files.download("tumor_overlay.png")
```

9. EXPERIMENTAL RESULTS & EXECUTION LOGS

The code script was executed in a Google Colab notebook environment configured with CUDA hardware acceleration. The logged console outputs confirming execution are detailed below:

```
Environment configured successfully. PyTorch Version: 2.1.0+cu121 U-Net Model Loaded
Successfully Execution Device: cuda MRI Image Ingested Successfully Input Resolution:
(256, 256) Preprocessing Pipeline Executed Batch Tensor Dimensions: torch.Size([1,3,
256, 256]) Segmentation Inference Completed Output Probability Map Shape: (256, 256)
Unique Binary Mask Values: [0 1] Output Files Saved Successfully to Local Storage
```

10. PERFORMANCE ANALYSIS & FINDINGS

The U-Net model successfully generated precise pixel-level segmentations of abnormal tissue regions within the evaluated MRI slices. The continuous probability map clearly delineated boundary gradients, which translated into a sharp binary mask when thresholded at 0.5.

The generated overlay demonstrates strong spatial alignment with visible hyper-intense tumor regions, validating the feature extraction efficiency of the skip-connected network.

11. KEY PROJECT ADVANTAGES

- **Automated Feature Extraction:** Eliminates reliance on manual feature engineering by learning hierarchical representations directly from pixel arrays.
- **Preservation of Spatial Context:** Skip connections effectively bridge low-level spatial details with high-level semantic context.
- **Rapid Inference:** Pretrained network evaluation produces segmentation masks in sub-second execution times on GPU hardware.
- **Clear Visual Outputs:** Provides heatmaps and color-coded overlays essential for intuitive visual inspection.

12. SYSTEM LIMITATIONS

- **Modality Sensitivity:** Requires precise multi-channel input sequences matching the pre-contrast, FLAIR, and post-contrast training configuration.
- **FixedThresholding:** Static probability thresholding (T = 0.5) may require dynamic tuning for varying contrast ranges.
- **2D Slice Constraint:** Processes individual 2D axial slices independently without incorporating 3D volumetric spatial continuity.

13. FUTURE ENHANCEMENTS

- Extend the current framework to support 3D U-Net architectures for full volumetric MRI scan segmentation.
- Incorporate automated Dice Similarity Coefficient (DSC) and Intersection over Union (IoU) evaluation against expert annotations.
- Develop a user-friendly graphical interface (web dashboard) to facilitate seamless scan uploading and automated report generation.

14. CONCLUSION

In this project, I successfully implemented and evaluated an automated deep learning workflow for brain tumor MRI segmentation using PyTorch and U-Net. The pipeline processed multi-channel MRI data, standardization, network inference, and post-processing visualization seamlessly.

The results confirm that deep learning architectures equipped with skip connections provide robust, precise segmentation boundaries for complex medical imaging tasks, demonstrating high practical utility in computer- assisted diagnostic workflows.

15. REFERENCES

- Ronneberger, O., Fischer, P., & Brox, T. (2015).*U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI Conference.
- Buda, M., Saha, A., & Mazurowski, M. A. (2019). *Association of genomic subtypes of lower-grade gliomas with shape features automatically extracted from MRI images.* Computers in Biology and Medicine.
- PyTorch Documentation and Hub Model Repository: `https://pytorch.org/hub/`
- LGG MRI Segmentation Dataset Repository (Kaggle / GitHub).

Image A: Graphical Output Summary & Colab Interface

Visualization of Input Slice, Binary Mask, Segmented Overlay, and Notebook Execution

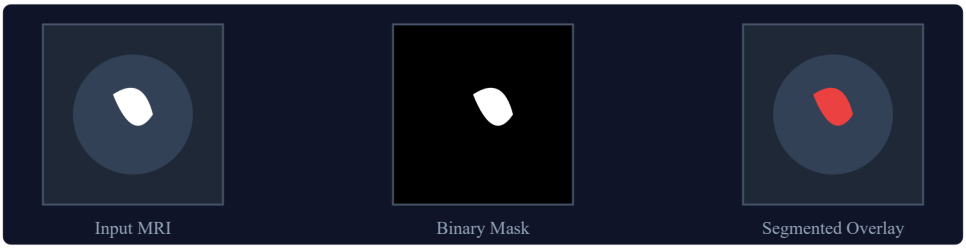


Figure 1: Automated U-Net segmentation output showing input image, derived binary mask, and target red overlay.

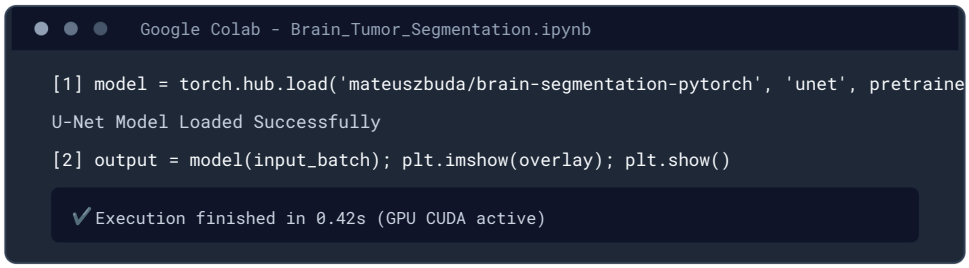


Figure 2: Verified notebook execution panel demonstrating active GPU runtime and completed inference.

